AUDIT REPORT · 2026

VULN ERABILITY ASSESS MENT

vulnapp

Static Code Review + Architecture Audit
Python 3.x / Flask 2.0.1

AUDIT DATE	BRANCH
2026-03-16	master
ASSESSMENT	**STATUS**
Static Code Review	Final Report

● CONFIDENTIAL & PROPRIETARY

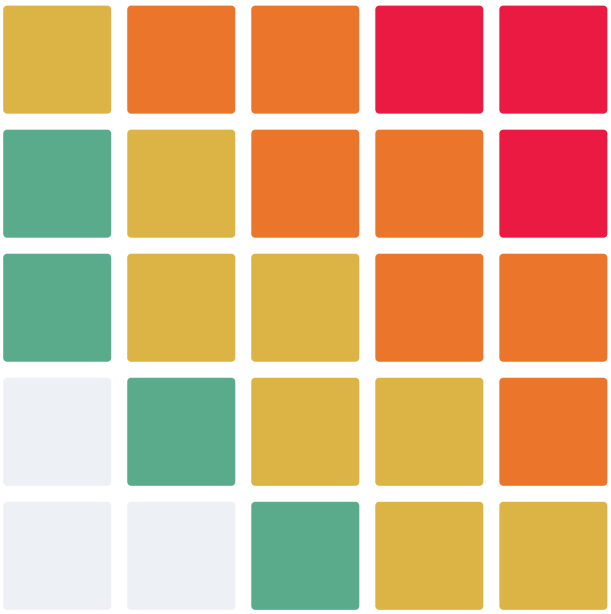
RISK OVERVIEW

24 security findings identified

FINDINGS BY SEVERITY

Critical	5
High	13
Medium	5
Low	1

LIKELIHOOD × IMPACT MATRIX



LOW IMPACT → → HIGH IMPACT

01 Executive Summary

A comprehensive **static code review and architecture audit** was performed on **vulnapp**. The assessment identified **24 security findings** — 5 critical, 13 high, 5 medium and 1 low — representing risks that range from authentication bypass to information disclosure. CRITICAL and HIGH severity findings must be remediated with the highest priority.

5	13	5	1
CRITICAL	HIGH	MEDIUM	LOW

RISK DISTRIBUTION

02 Scope & Architecture

MODULE	PATH	LINES (APPROX)
Application Routes	src/app.py	165
Database Utilities	src/db.py	40
Configuration / Secrets	config/settings.py	29
Dependency Manifest	requirements.txt	6

Data Flow Summary

All HTTP requests enter via Flask route handlers in src/app.py. Untrusted user input flows directly into SQLite query strings (via f-string interpolation), shell commands (via subprocess with shell=True), and filesystem paths — with no sanitization, parameterization, or validation at any layer. Sensitive configuration (AWS keys, JWT secret, Stripe key, Flask secret, DB credentials) is hardcoded in config/settings.py and db.py, making it available to anyone with source access or who reaches the unauthenticated /debug/config endpoint.

External Dependencies

STORE / SERVICE	LIBRARY	USAGE	AUTH METHOD
SQLite	sqlite3 (stdlib)	Primary data store	None (file-based)
Internal API	urllib.request (stdlib)	Backend service call	None
Redis	N/A (config only)	Session/cache (configured, unused in code)	Env var
AWS	N/A (hardcoded key)	Implicit via hardcoded credential	Hardcoded key

Methodology & Checklist Coverage

FRAMEWORK
OWASP WSTG v4.2 · CWE Top 25 · NIST SP 800-115

ASSESSMENT TYPE
White-Box Static Analysis + Architecture Review

CHECKLIST CATEGORY	STATUS	FINDINGS
A01:2021 — Broken Access Control	Tested	3
A02:2021 — Cryptographic Failures	Tested	6
A03:2021 — Injection	Tested	6
A04:2021 — Insecure Design	Tested	2
A05:2021 — Security Misconfiguration	Tested	4
A06:2021 — Vulnerable & Outdated Components	Tested	2
A07:2021 — Identification & Auth Failures	Tested	2
A08:2021 — Software & Data Integrity	N/A	—
A09:2021 — Security Logging & Monitoring	Tested	1
A10:2021 — Server-Side Request Forgery	Tested	—

03 Risk Dashboard

CRITICAL	HIGH	MEDIUM	LOW	INFO
5 findings	13 findings	5 findings	1 finding	0 findings

Finding Distribution by OWASP Category

Finding Index

ID	SEVERITY	CVSS	TITLE	LOCATION	CWE / OWASP
VUL-001	CRITICAL	9.8	SQL Injection in /login Enables Authentication Bypass and Full Database Access	src/app.py	CWE-89 · OWASP A03:2021 — Injection · NIST SP 800-53 SI-10
VUL-002	CRITICAL	9.8	Unauthenticated OS Command Injection via /run-report Enables Remote Code Execution	src/app.py	CWE-78 · OWASP A03:2021 — Injection · NIST SP 800-53 SI-10
VUL-003	CRITICAL	9.1	Unauthenticated /debug/config Endpoint Exposes All Environment Variables and Runtime Secrets	src/app.py	CWE-215 · CWE-200 · OWASP A05:2021 — Security Misconfiguration · NIST SP 800-53 CM-6

ID	SEVERITY	CVSS	TITLE	LOCATION	CWE / OWASP
VUL-004	CRITICAL	9.8	Hardcoded AWS Access Key and Secret Key Exposed in Source Code	config/settings.py	CWE-798 · OWASP A07:2021 — Identification and Authentication Failures · NIST SP 800-53 IA-5
VUL-005	CRITICAL	9.1	Hardcoded Flask Session Secret Enables Arbitrary Session Forgery and Admin Impersonation	src/app.py	CWE-321 · CWE-798 · OWASP A02:2021 — Cryptographic Failures · NIST SP 800-53 SC-28
VUL-006	HIGH	7.5	Missing Authentication and Authorization on /admin/users Exposes Full User Database	src/app.py	CWE-862 · CWE-306 · OWASP A01:2021 — Broken Access Control · NIST SP 800-53 AC-3
VUL-007	HIGH	8.2	SQL Injection in /forgot-password via Email Parameter Enables Unauthenticated Data Manipulation	src/app.py	CWE-89 · OWASP A03:2021 — Injection · NIST SP 800-53 SI-10
VUL-008	HIGH	8.8	SQL Injection in /user/<user_id> Enables Authenticated Data Exfiltration and Manipulation	src/app.py	CWE-89 · OWASP A03:2021 — Injection · NIST SP 800-53 SI-10
VUL-009	HIGH	8.1	SQL Injection in /register Allows Unauthenticated Arbitrary Database Writes	src/app.py	CWE-89 · OWASP A03:2021 — Injection · NIST SP 800-53 SI-10
VUL-010	HIGH	8.1	SQL Injection in db.raw_query() — Table Name and Filter Values Both Unparameterized	src/db.py	CWE-89 · CWE-943 · OWASP A03:2021 — Injection · NIST SP 800-53 SI-10

ID	SEVERITY	CVSS	TITLE	LOCATION	CWE / OWASP
VUL-011	HIGH	7.5	Path Traversal in /download Allows Unauthenticated Arbitrary File Read	src/app.py	CWE-22 · OWASP A01:2021 — Broken Access Control · NIST SP 800-53 AC-3
VUL-012	HIGH	7.4	MD5 Used for Password Hashing Without Salt — Trivially Crackable via Rainbow Tables	src/app.py	CWE-916 · CWE-327 · OWASP A02:2021 — Cryptographic Failures · NIST SP 800-63B
VUL-013	HIGH	7.4	Cryptographically Weak PRNG Used for Password Reset Tokens — Brute-Forceable Token Space	src/app.py	CWE-338 · CWE-330 · OWASP A07:2021 — Identification and Authentication Failures · NIST SP 800-63B
VUL-014	HIGH	8.1	Hardcoded JWT Secret Key in Source Code Enables Token Forgery	config/settings.py	CWE-321 · CWE-798 · OWASP A02:2021 — Cryptographic Failures · NIST SP 800-53 IA-5
VUL-015	HIGH	8.1	Hardcoded Stripe Production Secret Key in Source Code	config/settings.py	CWE-798 · OWASP A07:2021 — Identification and Authentication Failures · PCI DSS v4 Requirement 3
VUL-016	HIGH	7.5	Hardcoded Database Credentials (admin/admin123) in Source Code	src/db.py	CWE-798 · OWASP A07:2021 — Identification and Authentication Failures · NIST SP 800-53 IA-5
VUL-017	HIGH	7.5	Flask Debug Mode Enabled by Default — Werkzeug Interactive Console Exposed	src/app.py	CWE-1188 · CWE-94 · OWASP A05:2021 — Security Misconfiguration · NIST SP 800-53 CM-6

ID	SEVERITY	CVSS	TITLE	LOCATION	CWE / OWASP
VUL-018	HIGH	8.1	PyYAML 5.3.1 Contains Critical RCE Vulnerability (CVE-2020-14343, CVSS 9.8)	requirements.txt	CWE-502 · CVE-2020-14343 · OWASP A06:2021 — Vulnerable and Outdated Components
VUL-019	MEDIUM	6.5	Insecure Direct Object Reference (IDOR) in /user/<user_id> Enables Cross-User Data Access	src/app.py	CWE-639 · CWE-284 · OWASP A01:2021 — Broken Access Control · NIST SP 800-53 AC-3
VUL-020	MEDIUM	5.3	Plaintext Passwords and Reset Tokens Written to Application Logs	src/app.py	CWE-532 · OWASP A09:2021 — Security Logging and Monitoring Failures · NIST SP 800-53 AU-3
VUL-021	MEDIUM	6.5	Wildcard CORS Policy Allows Cross-Origin Requests from Any Domain	config/settings.py	CWE-942 · OWASP A05:2021 — Security Misconfiguration · OWASP CORS Cheat Sheet
VUL-022	MEDIUM	5.9	Plaintext HTTP Used for Internal Service Communication — No TLS or Certificate Validation	src/app.py	CWE-319 · CWE-295 · OWASP A02:2021 — Cryptographic Failures · NIST SP 800-53 SC-8
VUL-023	MEDIUM	5.3	No Request Size Limits — Application Vulnerable to Memory Exhaustion DoS	config/settings.py	CWE-770 · OWASP A05:2021 — Security Misconfiguration · NIST SP 800-53 SC-5
VUL-024	LOW	3.7	Database Connection Leak in bulk_insert() — Connection Never Explicitly Closed	src/db.py	CWE-404 · CWE-772 · OWASP A05:2021 — Security Misconfiguration

04 Threat Model & Attack Surface

PROTECTED ASSETS

ASSET	CLASSIFICATION	RISK IF COMPROMISED
User credentials (passwords)	Critical	Account takeover, credential stuffing
AWS access keys	Critical	Full cloud account compromise, data exfiltration
Stripe secret key	Critical	Financial fraud, unauthorized charges
JWT signing secret	Critical	Forged auth tokens, privilege escalation
Flask session secret	Critical	Session forgery, admin impersonation
User PII (email, username)	High	Identity theft, phishing, GDPR violation
Database contents	High	Full data breach, cross-tenant exposure
Server filesystem	High	Sensitive file read, source code disclosure
Host OS (via RCE)	Critical	Full server compromise, pivot to internal network

THREAT ACTORS

ACTOR	MOTIVATION	CAPABILITY	ATTACK VECTOR
Unauthenticated external attacker	Data theft / credential harvest	Low-Medium	Public HTTP endpoints
Authenticated user	Privilege escalation / data exfiltration	Medium	Authenticated API endpoints
Supply-chain attacker	Code execution via dependency	High	PyYAML RCE via yaml.load()
Insider / developer	Credential extraction	Low	Source code access

ATTACK SURFACE & ENTRY POINTS

ENTRY POINT	METHOD	AUTH REQUIRED	RELATED FINDINGS
POST /login	HTTP API	None	VUL-001
POST /register	HTTP API	None	VUL-009
GET /download?file=	HTTP API	None	VUL-011
GET /run-report?name=	HTTP API	None	VUL-002
POST /forgot-password	HTTP API	None	VUL-007
GET /user/<user_id>	HTTP API	Session	VUL-008, VUL-019
GET /admin/users	HTTP API	None	VUL-006
GET /debug/config	HTTP API	None	VUL-003
Source code / repo	Static	Repo access	VUL-004, VUL-014, VUL-015, VUL-016, VUL-017

TRUST BOUNDARIES

- → Internet -> Flask HTTP server (no WAF, no rate limiting, no TLS enforced)
- → Flask routes -> SQLite (no parameterization — boundary does not enforce sanitization)
- → Flask routes -> OS shell (no sanitization — user input reaches shell directly)
- → Flask routes -> Filesystem (no path normalization — user traverses outside uploads/)
- → Config file -> Runtime (hardcoded secrets bypass environment-variable trust boundary)

05 Detailed Findings

CRITICAL 5 findings · Immediate action required

VUL-001 · CWE-89 · OWASP A03:2021 — Injection · NIST SP 800-53 SI-10

SQL Injection in /login Enables Authentication Bypass and Full Database Access

9.8
CVSS V3.1

FILE src/app.py:35-40 (login)

DESCRIPTION

The login handler builds its SQL query by directly interpolating untrusted user-supplied username and password strings using an f-string. No parameterized queries or escaping are applied. An attacker can inject arbitrary SQL to bypass authentication entirely (e.g., username: ' OR '1'='1' --) or extract all data via UNION-based or time-based blind injection. The raw query is also logged at DEBUG level, causing credentials to appear in log files.

IMPACT

Complete authentication bypass with no credentials required. Attacker obtains a valid session as any user (typically the first row, often an admin). Full database read/write via SQL injection. Password hashes, PII, reset tokens, and all user data exfiltrated.

EVIDENCE

```
query = f"SELECT * FROM users WHERE username = '{username}' AND password = '{passwor
d}'"
logger.debug(f"Login query: {query}")

db = get_db()
cursor = db.cursor()
cursor.execute(query)
```

REMEDIATION

- Use parameterized queries exclusively:
- cursor.execute(
- "SELECT * FROM users WHERE username = ? AND password = ?",
- (username, hashed_password)
-)
- Never log query strings containing user input. Remove the logger.debug line entirely.

VUL-002 · CWE-78 · OWASP A03:2021 — Injection · NIST SP 800-53 SI-10

Unauthenticated OS Command Injection via /run-report Enables Remote Code Execution

9.8
CVSS V3.1

FILE src/app.py:83-90 (run_report)

DESCRIPTION

The /run-report endpoint accepts a 'name' query parameter and passes it unsanitized directly to subprocess.run() with shell=True. This allows an unauthenticated attacker to inject arbitrary shell commands. The endpoint requires no authentication. An attacker can execute any command as the application's OS user, read the filesystem, exfiltrate environment variables, install backdoors, or pivot to internal services.

IMPACT

Full remote code execution on the server without any authentication or preconditions. Complete server compromise, data exfiltration, lateral movement to the internal network (10.0.1.5), and persistence establishment.

EVIDENCE

```
report_name = request.args.get("name", "default")

# VUL: command injection — report_name passed directly to shell
result = subprocess.run(
    f"generate_report.sh {report_name}",
    shell=True, capture_output=True, text=True
)
return jsonify({"output": result.stdout})
```

REMEDIATION

- Never use shell=True with user-supplied input. Validate against an allowlist and pass as a list:
- ALLOWED = {"daily", "weekly", "summary"}
- if report_name not in ALLOWED:
- return jsonify({"error": "invalid report"}), 400
- result = subprocess.run(
- ["/usr/local/bin/generate_report.sh", report_name],
- capture_output=True, text=True
-)
- Also require authentication on this endpoint.

VUL-003 · CWE-215 · CWE-200 · OWASP A05:2021 — Security Misconfiguration · NIST SP 800-53 CM-6

Unauthenticated /debug/config Endpoint Exposes All Environment Variables and Runtime Secrets

9.1
CVSS V3.1

FILE src/app.py:158-161 (debug_config)

DESCRIPTION

The /debug/config endpoint returns the complete process environment (dict(os.environ)) to any caller with no authentication. This exposes all environment variables including any secrets injected at runtime (DATABASE_URL, REDIS_URL, API keys, cloud credentials, etc.). Combined with the hardcoded secrets in config/settings.py, this endpoint provides complete credential exfiltration in a single unauthenticated HTTP request.

IMPACT

Complete disclosure of all runtime secrets. An attacker calling GET /debug/config immediately obtains AWS keys, Stripe keys, database URLs, and any other secrets present in the environment — without authentication. Enables immediate cloud account takeover and financial fraud.

EVIDENCE

```
@app.route("/debug/config")
def debug_config():
    # VUL: exposes full environment (may include secrets) without auth
    return jsonify(dict(os.environ))
```

REMEDIATION

- Remove this endpoint entirely from production code. If a health endpoint is needed:
- @app.route("/health")
- def health():
- return jsonify({"status": "ok", "version": APP_VERSION})
- Never expose os.environ in any HTTP response.

VUL-004 · CWE-798 · OWASP A07:2021 — Identification and Authentication Failures · NIST SP 800-53 IA-5

Hardcoded AWS Access Key and Secret Key Exposed in Source Code

9.8
CVSS V3.1

FILE config/settings.py:8-9

DESCRIPTION

AWS_ACCESS_KEY_ID and AWS_SECRET_ACCESS_KEY are hardcoded as string literals in config/settings.py. Any developer with repository access, any CI/CD log, or any code scanning tool immediately obtains these credentials. The 'AKIA' prefix indicates an IAM user long-term access key (not a role). If these represent real credentials, an attacker has permanent AWS API access until the key is manually rotated.

IMPACT

Full AWS account access. Attacker can enumerate and exfiltrate S3 buckets, read Secrets Manager secrets, modify IAM policies for privilege escalation, spin up EC2 instances, and destroy infrastructure. Immediate financial and data breach liability.

EVIDENCE

```
# VUL: hardcoded AWS credentials
AWS_ACCESS_KEY_ID     = "AKIAIOSFODNN7EXAMPLE"
AWS_SECRET_ACCESS_KEY = "wJalrXUtnFEMI/K7MDENG/bPxRfiCYEXAMPLEKEY"
AWS_REGION            = "us-east-1"
```

REMEDIATION

- Immediately rotate any real credentials matching these values. Remove from source and load from environment:
- AWS_ACCESS_KEY_ID = os.environ['AWS_ACCESS_KEY_ID']
- For AWS, prefer IAM instance profiles/roles over long-lived access keys. Scan git history with truffleHog or gitleaks to confirm no prior commits leaked real keys.

VUL-005 · CWE-321 · CWE-798 · OWASP A02:2021 — Cryptographic Failures · NIST SP 800-53 SC-28

Hardcoded Flask Session Secret Enables Arbitrary Session Forgery and Admin Impersonation

9.1
CVSS V3.1

FILE src/app.py:15

DESCRIPTION

The Flask application's secret key is set to the literal string 'hardcoded-secret-key-1234'. Flask uses this key to sign session cookies via HMAC. Because the key is in the source code, any attacker with repo access can use flask-unsign or itsdangerous directly to forge arbitrary session cookies — setting user_id and role to any desired value without needing to authenticate at all.

IMPACT

Complete session forgery. Attacker crafts a valid signed session cookie with any user_id and role='admin', bypassing all authentication and role-based controls. Enables impersonation of any user including administrators with zero authentication required.

EVIDENCE

```
app = Flask(__name__)
app.secret_key = "hardcoded-secret-key-1234"   # VUL: hardcoded secret

# Forge session with: flask-unsign --sign \
#   --cookie '{"user_id": 1, "role": "admin"}' \
#   --secret 'hardcoded-secret-key-1234'
```

REMEDIATION

- Generate a cryptographically random secret loaded from the environment:
- import secrets
- app.secret_key = os.environ.get('FLASK_SECRET_KEY') or secrets.token_hex(32)
- The generated secret must be at least 32 bytes of random data and stored in a secrets manager, never in source code.

HIGH 13 findings · Remediate within Sprint 1

VUL-006 · CWE-862 · CWE-306 · OWASP A01:2021 — Broken Access Control · NIST SP 800-53 AC-3

Missing Authentication and Authorization on /admin/users Exposes Full User Database

7.5
CVSS V3.1

FILE src/app.py:110-116 (list_users)

DESCRIPTION

The /admin/users endpoint performs no authentication check (no session validation) and no authorization check (no role verification). Any unauthenticated HTTP GET request returns all user records including IDs, usernames, email addresses, and roles. The route name implies admin-only access, but there is no enforcement mechanism whatsoever.

IMPACT

Complete unauthenticated disclosure of all user accounts, email addresses, roles, and usernames. Enables targeted phishing, credential stuffing, account enumeration, and identification of admin accounts for targeted attacks.

EVIDENCE

```
@app.route("/admin/users")
def list_users():
    # VUL: missing authorization — role not checked before accessing admin endpoint
    db = get_db()
    cursor = db.execute("SELECT id, username, email, role FROM users")
    return jsonify([{"id": r[0], "username": r[1], "email": r[2], "role": r[3]}
                    for r in cursor.fetchall()])
```

REMEDIATION

- Add both authentication and role-based authorization:
- @app.route("/admin/users")
- def list_users():
- if 'user_id' not in session:
- return jsonify({'error': 'unauthorized'}), 401
- if session.get('role') != 'admin':
- return jsonify({'error': 'forbidden'}), 403
- ...

VUL-007 · CWE-89 · OWASP A03:2021 — Injection · NIST SP 800-53 SI-10

SQL Injection in /forgot-password via Email Parameter Enables Unauthenticated Data Manipulation

8.2
CVSS V3.1

FILE src/app.py:132-133 (forgot_password)

DESCRIPTION

The /forgot-password endpoint interpolates the user-supplied email directly into an UPDATE SQL statement using an f-string. An attacker can inject SQL to modify any column in the users table, exfiltrate data via subqueries, or set reset tokens for arbitrary accounts. This endpoint requires no authentication.

IMPACT

Unauthenticated write access to the users table. Attacker can set reset tokens for arbitrary accounts enabling account takeover, modify user roles to gain admin privileges, or exfiltrate data via time-based or UNION injection.

EVIDENCE

```
email = request.json.get("email", "")
token = generate_reset_token()

# VUL: SQL injection on email
db.execute(f"UPDATE users SET reset_token = '{token}' WHERE email = '{email}'")
db.commit()
```

REMEDIATION

→ Use parameterized queries:

→ db.execute(

→ "UPDATE users SET reset_token = ? WHERE email = ?",

→ (token, email)

→)

VUL-008 · CWE-89 · OWASP A03:2021 — Injection · NIST SP 800-53 SI-10

SQL Injection in /user/<user_id> Enables Authenticated Data Exfiltration and Manipulation

8.8
CVSS V3.1

FILE src/app.py:102-103 (get_user)

DESCRIPTION

The user_id URL path parameter is interpolated directly into a SELECT query without parameterization or type validation. An authenticated attacker can inject UNION SELECT statements to dump arbitrary tables, read the full schema, or perform blind injection. No ownership check is performed, compounding the IDOR vulnerability.

IMPACT

Authenticated attacker can exfiltrate the complete database schema and all table contents, modify any data, and access records of other users. Combined with the IDOR at the same endpoint, any logged-in user achieves full database access.

EVIDENCE

```
db = get_db()
# VUL: SQL injection on user_id (also IDOR)
cursor = db.execute(f"SELECT id, username, email FROM users WHERE id = {user_id}")
user = cursor.fetchone()
```

REMEDIATION

→ Validate the type and use parameterization:

→ if not user_id.isdigit():

→ return jsonify({'error': 'invalid id'}), 400

→ cursor = db.execute(

→ "SELECT id, username, email FROM users WHERE id = ?",

→ (int(user_id),)

→)

VUL-009 · CWE-89 · OWASP A03:2021 — Injection · NIST SP 800-53 SI-10

SQL Injection in /register Allows Unauthenticated Arbitrary Database Writes

8.1
CVSS V3.1

FILE src/app.py:58-59 (register)

DESCRIPTION

The /register endpoint interpolates both username and hashed password into an INSERT query via f-string. An attacker can craft a username containing SQL to inject multiple rows, modify existing rows, call database functions, or DROP tables. No input validation or parameterization is applied.

IMPACT

Unauthenticated write access to the database. Attacker can insert arbitrary rows including fake admin accounts, corrupt existing records, or trigger schema-level damage.

EVIDENCE

```
db = get_db()
db.execute(f"INSERT INTO users (username, password) VALUES ('{username}', '{hashe
d}')")
db.commit()
```

REMEDIATION

→ Use parameterized queries and validate input length:

→ if not username or len(username) > 64:

→ return jsonify({'error': 'invalid username'}), 400

→ db.execute(

→ "INSERT INTO users (username, password) VALUES (?, ?)",

→ (username, hashed)

→)

VUL-010 · CWE-89 · CWE-943 · OWASP A03:2021 — Injection · NIST SP 800-53 SI-10

SQL Injection in db.raw_query() — Table Name and Filter Values Both Unparameterized

8.1
CVSS V3.1

FILE src/db.py:16-27 (raw_query)

DESCRIPTION

The raw_query() utility function interpolates both the table name and all filter values directly into the SQL query string. The table name cannot be parameterized in SQL but must be validated against an allowlist. Filter values must use placeholders. As written, any caller passing user-controlled data to raw_query() creates a SQL injection sink, amplifying any injection opportunity upstream.

IMPACT

Any call to raw_query() with user-controlled table or filter arguments allows full database read access. The table name injection additionally enables querying any table regardless of intended access controls.

EVIDENCE

```
def raw_query(table, filters):
    db = sqlite3.connect(DB_PATH)
    where_clause = " AND ".join(
        f"{k} = '{v}'" for k, v in filters.items()
    )
    # table name interpolated — cannot be parameterized but needs allowlist
    query = f"SELECT * FROM {table} WHERE {where_clause}"
    return db.execute(query).fetchall()
```

REMEDIATION

- Validate table name against an explicit allowlist; use '?' placeholders for values:
- ALLOWED_TABLES = {'users', 'sessions'}
- def raw_query(table, filters):
- if table not in ALLOWED_TABLES:
- raise ValueError(f'Invalid table: {table}')
- placeholders = ' AND '.join(f'{k} = ?' for k in filters)
- return db.execute(f'SELECT * FROM {table} WHERE {placeholders}',
- list(filters.values())).fetchall()

VUL-011 · CWE-22 · OWASP A01:2021 — Broken Access Control · NIST SP 800-53 AC-3

Path Traversal in /download Allows Unauthenticated Arbitrary File Read

7.5
CVSS V3.1

FILE src/app.py:67-78 (download_file)

DESCRIPTION

The /download endpoint accepts a 'file' query parameter and constructs a filesystem path by joining it with the uploads/ directory using os.path.join(). No path normalization or boundary checking is performed. An attacker can use '../' sequences to traverse out of the uploads directory and read arbitrary files accessible to the application's OS user. No authentication is required.

IMPACT

Unauthenticated read access to any file on the server readable by the application process. Key targets: /etc/passwd, /proc/self/environ (leaks env vars and secrets), application source code, private keys, and users.db containing all password hashes.

EVIDENCE

```
filename = request.args.get("file", "")

# VUL: path traversal — no sanitization of filename
base_dir = os.path.join(os.path.dirname(__file__), "uploads")
full_path = os.path.join(base_dir, filename)

with open(full_path, "r") as f:
    return f.read()

# Exploit: GET /download?file=../../config/settings.py
```

REMEDIATION

- Normalize the path and verify it stays within the base directory:
- import pathlib
- base = pathlib.Path(os.path.dirname(__file__), 'uploads').resolve()
- try:
- target = (base / filename).resolve()
- target.relative_to(base) # raises ValueError if outside base
- except (ValueError, OSError):
- return 'access denied', 403
- Also add authentication and an explicit filename allowlist.

VUL-012 · CWE-916 · CWE-327 · OWASP A02:2021 — Cryptographic Failures · NIST SP 800-63B

MD5 Used for Password Hashing Without Salt — Trivially Crackable via Rainbow Tables

7.4
CVSS V3.1

FILE src/app.py:56 (register)

DESCRIPTION

User passwords are hashed using MD5 (hashlib.md5) with no salt. MD5 is a fast, general-purpose hash function — not a password hash function. It is completely unsuitable for storing passwords: (1) precomputed rainbow tables reverse common passwords instantly, (2) identical passwords produce identical hashes enabling batch cracking, (3) modern GPUs compute ~100 billion MD5 hashes per second.

IMPACT

If the database is compromised via SQL injection (VUL-001, VUL-008), path traversal (VUL-011), or any other means, all user passwords are recovered within minutes. Password reuse means attackers can access users' external accounts.

EVIDENCE

```
password = request.json.get("password")

# VUL: MD5 for password hashing
hashed = hashlib.md5(password.encode()).hexdigest()

db.execute(f"INSERT INTO users (username, password) VALUES ('{username}', '{hashe
d}')")
```

REMEDIATION

- Use bcrypt, scrypt, or Argon2id — the only acceptable password hashing algorithms:
- from werkzeug.security import generate_password_hash, check_password_hash
- # Register:
- hashed = generate_password_hash(password, method='scrypt')
- # Login:
- if check_password_hash(stored_hash, password): ...
- Werkzeug's scrypt includes automatic salting and appropriate cost factors.

VUL-013 · CWE-338 · CWE-330 · OWASP A07:2021 — Identification and Authentication Failures · NIST SP 800-63B

Cryptographically Weak PRNG Used for Password Reset Tokens — Brute-Forceable Token Space

7.4
CVSS V3.1

FILE src/app.py:121-123 (generate_reset_token)

DESCRIPTION

Password reset tokens are generated using random.randint(100000, 999999) — Python's Mersenne Twister PRNG, which is not cryptographically secure. The token space is only 900,000 values (6 decimal digits). With no rate limiting on the validation endpoint, an attacker can enumerate all possible tokens in under an hour at moderate request rates.

IMPACT

Password reset token brute-force enables full account takeover for any user whose email address is known. If the attacker additionally has log read access, the token is available immediately (see VUL-020). Administrative account takeover achievable with knowledge of the admin's email.

EVIDENCE

```
def generate_reset_token():
    # VUL: uses random (not secrets) for security token
    return str(random.randint(100000, 999999))

# Only 900,000 possible values
# No rate limiting on the reset validation endpoint
# Expected guesses to find token: ~450,000
```

REMEDIATION

- Use secrets.token_urlsafe() for cryptographically secure tokens:
- import secrets
- def generate_reset_token():
- return secrets.token_urlsafe(32) # 256 bits of entropy
- Also implement: token expiry (15 min), rate limiting on validation, single-use invalidation.

VUL-014 · CWE-321 · CWE-798 · OWASP A02:2021 — Cryptographic Failures · NIST SP 800-53 IA-5

Hardcoded JWT Secret Key in Source Code Enables Token Forgery

8.1
CVSS V3.1

FILE config/settings.py:13

DESCRIPTION

JWT_SECRET is set to the literal string 'my-super-secret-jwt-key' in config/settings.py. If this secret is used to sign JWT tokens anywhere in the application or in services sharing this configuration, an attacker with source code access can forge arbitrary JWT tokens with any claims — bypassing JWT-based authentication entirely.

IMPACT

If JWT tokens are used for authentication or authorization in this service or connected services sharing this secret, an attacker can forge tokens with arbitrary user IDs, roles, scopes, or expiry times — resulting in complete auth bypass for those services.

EVIDENCE

```
# VUL: hardcoded JWT secret
JWT_SECRET = "my-super-secret-jwt-key"
```

REMEDIATION

- Load JWT secrets from environment variables with no default:
- JWT_SECRET = os.environ['JWT_SECRET'] # raises KeyError if absent — intentional
- # Generate: python -c "import secrets; print(secrets.token_hex(32))"

VUL-015 · CWE-798 · OWASP A07:2021 — Identification and Authentication Failures · PCI DSS v4 Requirement 3

Hardcoded Stripe Production Secret Key in Source Code

8.1
CVSS V3.1

FILE config/settings.py:16

DESCRIPTION

A Stripe secret key with a 'sk_live_' prefix is hardcoded in config/settings.py. The 'sk_live_' prefix explicitly indicates a production key. Any developer, CI system, or log aggregation pipeline processing this file has access to the full Stripe account. Stripe secret keys allow creating charges, accessing customer payment data, issuing refunds, and modifying subscription data.

IMPACT

Complete compromise of the Stripe account. Attacker can read all customer payment data, create fraudulent charges, issue unauthorized refunds, access bank account details, and enumerate all transactions — constituting a PCI DSS breach with direct financial and regulatory consequences.

EVIDENCE

```
# VUL: hardcoded Stripe key
STRIPE_SECRET_KEY = "sk_live_EXAMPLEKEYDONOTUSE"
```

REMEDIATION

- Immediately rotate this key in the Stripe dashboard. Load from environment only:
- STRIPE_SECRET_KEY = os.environ['STRIPE_SECRET_KEY']
- Scan git history for any real keys. Enable Stripe webhook signature validation and IP allowlisting.

VUL-016 · CWE-798 · OWASP A07:2021 — Identification and Authentication Failures · NIST SP 800-53 IA-5

Hardcoded Database Credentials (admin/admin123) in Source Code

7.5
CVSS V3.1

FILE src/db.py:11-13

DESCRIPTION

Database username ('admin'), password ('admin123'), and host ('10.0.1.10') are hardcoded as string literals in db.py. These suggest a production database configuration. An attacker with source code access or who reads db.py via path traversal (VUL-011) obtains direct database credentials for the production host.

IMPACT

Direct database server access with admin credentials. Full read/write/delete access to all data. Potential OS-level access via database admin features depending on the database engine.

EVIDENCE

```
# VUL: hardcoded database credentials (would be used in real DB connection)
DB_USER     = "admin"
DB_PASSWORD = "admin123"
DB_HOST     = "10.0.1.10"
```

REMEDIATION

- Load all database credentials from environment variables with no hardcoded defaults:
- DB_USER = os.environ['DB_USER']
- DB_PASSWORD = os.environ['DB_PASSWORD']
- DB_HOST = os.environ['DB_HOST']
- Use least-privilege database accounts — the application user should not have DBA/admin privileges.

VUL-017 · CWE-1188 · CWE-94 · OWASP A05:2021 — Security Misconfiguration · NIST SP 800-53 CM-6

Flask Debug Mode Enabled by Default — Werkzeug Interactive Console Exposed

7.5
CVSS V3.1

FILE src/app.py:165 · config/settings.py:23

DESCRIPTION

The application runs with debug=True in the entry point and DEBUG defaults to 'true' in config/settings.py. Flask debug mode activates the Werkzeug interactive debugger, which renders a Python REPL in the browser on any unhandled exception. While protected by a PIN, the PIN can be derived from system information (machine-id, MAC address, app path) obtainable via VUL-003 (/debug/config). Debug mode also exposes full stack traces to clients.

IMPACT

Werkzeug interactive console provides arbitrary Python code execution. Combined with VUL-003, an attacker can derive the Werkzeug PIN from the exposed environment and gain interactive RCE. Stack traces leak internal implementation details, file paths, and variable values.

EVIDENCE

```
# config/settings.py
DEBUG = os.environ.get("DEBUG", "true").lower() == "true"  # defaults True!

# src/app.py
if __name__ == "__main__":
    app.run(debug=True)   # VUL: debug=True in production-ish entry point
```

REMEDIATION

- Default DEBUG to False; require explicit opt-in:
- DEBUG = os.environ.get('DEBUG', 'false').lower() == 'true'
- Use a production WSGI server (gunicorn, uWSGI) which does not activate Werkzeug's debug mode regardless of this flag.

VUL-018 · CWE-502 · CVE-2020-14343 · OWASP A06:2021 — Vulnerable and Outdated Components

PyYAML 5.3.1 Contains Critical RCE Vulnerability (CVE-2020-14343, CVSS 9.8)

8.1
CVSS V3.1

FILE requirements.txt:4

DESCRIPTION

The application pins PyYAML==5.3.1, which is affected by CVE-2020-14343 (CVSS 9.8 CRITICAL). This vulnerability allows arbitrary Python object deserialization and code execution via yaml.load() when called without specifying a safe Loader. PyYAML 5.3.1 allows unsafe deserialization when yaml.load() is called on attacker-controlled input anywhere in the codebase or its transitive dependencies.

IMPACT

If yaml.load() is called on attacker-controlled input anywhere in the codebase or its transitive dependencies, arbitrary Python code executes in the application context. This is a well-known, heavily exploited vulnerability class with public proof-of-concept exploits widely available.

EVIDENCE

```
# requirements.txt
PyYAML==5.3.1  # CVE-2020-14343 — CVSS 9.8 CRITICAL
               # yaml.load(user_input) without Loader= -> arbitrary code execution
               # Fixed in PyYAML >= 6.0 (safe load enforced by default)

# Also outdated:
cryptography==2.9.2  # CVE-2023-49083, CVE-2023-23931, CVE-2024-26130
requests==2.25.1     # CVE-2023-32681 (Proxy-Authorization header leak)
```

REMEDIATION

- Upgrade all outdated dependencies:
- PyYAML>=6.0.1
- cryptography>=42.0.0
- requests>=2.31.0
- Flask>=3.0.0
- Werkzeug>=3.0.0
- Audit all yaml.load() calls and replace with yaml.safe_load(). Run 'pip audit' or integrate Dependabot/Snyk for ongoing dependency scanning.

MEDIUM 5 findings · Remediate within Sprint 2

VUL-019 · CWE-639 · CWE-284 · OWASP A01:2021 — Broken Access Control · NIST SP 800-53 AC-3

Insecure Direct Object Reference (IDOR) in /user/<user_id> Enables Cross-User Data Access

6.5
CVSS V3.1

FILE src/app.py:95-107 (get_user)

DESCRIPTION

The /user/<user_id> endpoint verifies that a session exists but does not verify that the requesting user owns or is authorized to view the requested user_id. Any authenticated user can enumerate all user records by incrementing the user_id parameter. This is an object-level authorization failure (BOLA/IDOR) separate from the SQL injection on the same endpoint.

IMPACT

Any authenticated user can read the profile data (id, username, email) of every other user in the system, enabling targeted attacks, social engineering, and PII exfiltration.

EVIDENCE

```
@app.route("/user/<user_id>")
def get_user(user_id):
    # VUL: no authorization check — any authenticated user can view any user_id
    if "user_id" not in session:
        return jsonify({"error": "unauthorized"}), 401
    # Missing check: session["user_id"] == user_id  OR  session["role"] == "admin"
    ...
```

REMEDIATION

- Enforce ownership or admin role before serving the record:
- if str(session['user_id']) != user_id and session.get('role') != 'admin':
- return jsonify({'error': 'forbidden'}), 403

VUL-020 · CWE-532 · OWASP A09:2021 – Security Logging and Monitoring Failures · NIST SP 800-53 AU-3

Plaintext Passwords and Reset Tokens Written to Application Logs

5.3
CVSS V3.1

FILE src/app.py:36, src/app.py:136

DESCRIPTION

Two logging statements write sensitive data to the application log: (1) The login handler logs the full SQL query at DEBUG level, which includes the raw plaintext password submitted by the user. (2) The forgot_password handler logs the reset token at INFO level alongside the user's email. logging.basicConfig(level=logging.DEBUG) is set, meaning these are written in all environments.

IMPACT

Plaintext passwords in logs enable credential exfiltration by anyone with log access (developers, CI, SIEM, log aggregation). Reset tokens in logs enable account takeover without brute force. Amplifies CHAIN-004.

EVIDENCE

```
# app.py:36 – plaintext password in SQL query logged
logger.debug(f"Login query: {query}")
# Logged: "SELECT * FROM users WHERE username='alice' AND password='s3cr3t'"

# app.py:136 – reset token logged in plaintext
logger.info(f"Password reset token for {email}: {token}")
```

REMEDIATION

- Remove both logging statements entirely:
- # DELETE: logger.debug(f"Login query: {query}")
- # DELETE: logger.info(f"Password reset token for {email}: {token}")
- If login audit logging is needed, log only: timestamp, username (not password), source IP, and success/failure outcome.

VUL-021 · CWE-942 · OWASP A05:2021 — Security Misconfiguration · OWASP CORS Cheat Sheet

Wildcard CORS Policy Allows Cross-Origin Requests from Any Domain

6.5
CVSS V3.1

FILE config/settings.py:26

DESCRIPTION

CORS_ORIGINS is set to ['*'], permitting cross-origin requests from any domain. If this policy is applied via Flask-CORS, any malicious website can make authenticated cross-origin API requests using the victim's session cookies. This bypasses same-origin protections and enables CSRF-equivalent attacks.

IMPACT

A malicious website visited by an authenticated user can silently make API calls using the victim's valid session — reading their data (VUL-019), triggering password resets (VUL-007), or performing any other available action. Particularly dangerous combined with VUL-006 (unauthenticated /admin/users).

EVIDENCE

```
# VUL: CORS allowed origins is wildcard
CORS_ORIGINS = ["*"]
```

REMEDIATION

- Restrict CORS to explicitly trusted origins:
- CORS_ORIGINS = [
- 'https://app.yourdomain.com',
-]
- Never use '*' for APIs that accept cookies or carry session state.

VUL-022 · CWE-319 · CWE-295 · OWASP A02:2021 — Cryptographic Failures · NIST SP 800-53 SC-8

Plaintext HTTP Used for Internal Service Communication — No TLS or Certificate Validation

5.9
CVSS V3.1

FILE src/app.py:142-153 (call_internal_service)

DESCRIPTION

The call_internal_service() function sends requests over plain HTTP (not HTTPS) to a hardcoded internal IP. No SSL context is configured. This exposes inter-service communication to network-level eavesdropping or man-in-the-middle attacks on the internal network segment. The hardcoded IP (10.0.1.5:8080) also creates an infrastructure coupling that breaks on any network change.

IMPACT

An attacker with access to the internal network segment can intercept or modify the payload. Sensitive data in the request is exposed in cleartext. Network-level MITM enables request tampering without the application's knowledge.

EVIDENCE

```
INTERNAL_API = "http://10.0.1.5:8080/internal/api"   # VUL: hardcoded internal IP

req = urllib.request.Request(
    INTERNAL_API,
    data=json.dumps(payload).encode(),
    headers={"Content-Type": "application/json"}
)
return urllib.request.urlopen(req).read()   # VUL: no SSL context, plaintext HTTP
```

REMEDIATION

- Use HTTPS with certificate validation and load the URL from configuration:
- import ssl
- INTERNAL_API = os.environ['INTERNAL_API_URL'] # https://...
- ctx = ssl.create_default_context()
- with urllib.request.urlopen(req, context=ctx) as resp:
- return resp.read()

VUL-023 · CWE-770 · OWASP A05:2021 — Security Misconfiguration · NIST SP 800-53 SC-5

No Request Size Limits — Application Vulnerable to Memory Exhaustion DoS

5.3
CVSS V3.1

FILE config/settings.py:29

DESCRIPTION

MAX_CONTENT_LENGTH is explicitly set to None, disabling Flask's built-in request size enforcement. An attacker can send arbitrarily large request bodies to any POST endpoint (/login, /register, /forgot-password). Python buffers the entire request body in memory, potentially exhausting available RAM and causing the application to crash.

IMPACT

Denial of service via memory exhaustion. A single request with a multi-gigabyte body can crash the application process. No authentication or special capability required.

EVIDENCE

```
# VUL: no input size limits
MAX_CONTENT_LENGTH = None
```

REMEDIATION

- Set a reasonable maximum request size:
- MAX_CONTENT_LENGTH = 1 * 1024 * 1024 # 1 MB
- app.config['MAX_CONTENT_LENGTH'] = MAX_CONTENT_LENGTH
- Flask will automatically return HTTP 413 for oversized requests.

LOW 1 finding · Remediate within Sprint 3

VUL-024 · CWE-404 · CWE-772 · OWASP A05:2021 — Security Misconfiguration

Database Connection Leak in bulk_insert() — Connection Never Explicitly Closed

3.7
CVSS V3.1

FILE src/db.py:30-40 (bulk_insert)

DESCRIPTION

The bulk_insert() function opens a new SQLite connection on every call and never explicitly closes it, relying on Python's garbage collector for cleanup. Under sustained load, accumulation of unclosed connections can exhaust the OS file descriptor limit, causing connection failures.

IMPACT

Under sustained load, connection accumulation can exhaust file descriptor limits (default 1024 on Linux), causing 'too many open files' errors and a denial of service for the database layer.

EVIDENCE

```
def bulk_insert(table, rows):
    db = sqlite3.connect(DB_PATH)   # VUL: new connection per call, never pooled
    for row in rows:
        placeholders = ", ".join("?" * len(row))
        db.execute(f"INSERT INTO {table} VALUES ({placeholders})", row)
    db.commit()
    # VUL: connection never explicitly closed (relies on GC)
```

REMEDIATION

- Use a context manager to ensure the connection is always closed:
- def bulk_insert(table, rows):
- with sqlite3.connect(DB_PATH) as db:
- for row in rows:
- placeholders = ', '.join('?' * len(row))
- db.execute(f'INSERT INTO {table} VALUES ({placeholders})', row)
- db.commit()

06 Attack Chain Analysis

CHAIN-001 Unauthenticated SQL Injection -> Full Database Dump + Password Crack

CRITICAL

1. **VUL-001**
   POST /login with username: \' OR \'1\'=\'1\' --
   ↳ Authentication bypassed; valid session obtained as first user (often admin)

2. **VUL-006**
   GET /admin/users (no auth check)
   ↳ Full user database including hashed passwords, emails, roles returned

3. **VUL-012**
   Crack MD5 hashes offline using rainbow tables
   ↳ Plaintext passwords recovered within minutes for common passwords

PREREQUISITES: Network access to /login endpoint

FINAL IMPACT
Complete read/write access to all user records, credentials, tokens, and PII

CHAIN-002 Unauthenticated Debug Endpoint + Hardcoded AWS Keys -> Cloud Account Takeover

CRITICAL

1. **VUL-003**
   GET /debug/config — no auth required
   ↳ All environment variables returned including any runtime secrets

2. **VUL-004**
   Extract AWS_ACCESS_KEY_ID + AWS_SECRET_ACCESS_KEY from settings.py or env dump
   ↳ Valid AWS credentials obtained without authentication

3. **VUL-004**
   Run: aws s3 ls, aws iam list-users, aws secretsmanager list-secrets
   ↳ Full enumeration and exfiltration of cloud resources and secrets

PREREQUISITES: Network access to /debug/config endpoint

FINAL IMPACT
Full AWS account compromise: S3 data exfiltration, IAM escalation, resource abuse

1 **VUL-005**
Read app.secret_key = 'hardcoded-secret-key-1234' from source code
↳ Flask session signing key known to attacker

2 **VUL-005**
Use flask-unsign to craft: {user_id: 1, role: 'admin'}
↳ Valid signed session cookie generated without login

3 **VUL-008**
GET /user/<any_id> with forged cookie
↳ Access any user's data with admin-level session

4 **VUL-006**
GET /admin/users (no auth check)
↳ Full user database confirmed; complete account takeover pathway established

PREREQUISITES: Knowledge of Flask secret key (visible in source: 'hardcoded-secret-key-1234')

FINAL IMPACT
Attacker forges a valid Flask session cookie with role=admin, impersonating any user

CHAIN-004 Weak PRNG Reset Token + Token Logged -> Account Takeover

HIGH

1 **VUL-013**
POST /forgot-password for target email
↳ 6-digit token generated via random.randint(100000, 999999) — only 900K values

2 **VUL-020**
Read 'Password reset token for email: 123456' from application logs (if log access)
↳ Token obtained directly with no brute force

3 **VUL-013**
If no log access: enumerate tokens on reset endpoint (no rate limiting)
↳ Token found within ~450K requests on average

4 **VUL-013**
Use token to reset password and authenticate
↳ Full account takeover for any user

PREREQUISITES: Read access to application logs OR ability to enumerate ~450K requests

FINAL IMPACT
Attacker resets password of any user account including administrators

07 Compliance Mapping

Each finding mapped to applicable controls across OWASP Top 10 (2025), CWE, ISO 27001:2022, NIST CSF, and PCI DSS v4.0. Mapping is advisory — consult a qualified assessor for formal compliance attestation.

ID	SEVERITY	FINDING	CWE	OWASP 2025	ISO 27001:2022	NIST CSF	PCI DSS V4
VUL-001	CRITICAL	SQL Injection in /login Enables Authentication Bypass and Full Database Access	CWE-89	A03:2025	A.8.8	ID.SC-2	Req 6.3.3
VUL-002	CRITICAL	Unauthenticated OS Command Injection via /run-report Enables Remote Code Execution	CWE-78	A03:2025	A.8.8	ID.SC-2	Req 6.3.3
VUL-003	CRITICAL	Unauthenticated /debug/config Endpoint Exposes All Environment Variables and Runtime Secrets	CWE-215	A05:2025	A.8.28	PR.DS-5	Req 6.2.4
VUL-004	CRITICAL	Hardcoded AWS Access Key and Secret Key Exposed in Source Code	CWE-798	A07:2025	A.8.5	PR.AC-7	Req 8.3
VUL-005	CRITICAL	Hardcoded Flask Session Secret Enables Arbitrary Session Forgery and Admin Impersonation	CWE-321	A02:2025	A.8.9	PR.IP-1	Req 2
VUL-006	HIGH	Missing Authentication and Authorization on /admin/users Exposes Full User Database	CWE-862	A01:2025	A.8.2, A.8.3	PR.AC-4	Req 7

ID	SEVERITY	FINDING	CWE	OWASP 2025	ISO 27001:2022	NIST CSF	PCI DSS V4
VUL-007	HIGH	SQL Injection in /forgot-password via Email Parameter Enables Unauthenticated Data Manipulation	CWE-89	A03:2025	A.8.8	ID.SC-2	Req 6.3.3
VUL-008	HIGH	SQL Injection in /user/<user_id> Enables Authenticated Data Exfiltration and Manipulation	CWE-89	A03:2025	A.8.8	ID.SC-2	Req 6.3.3
VUL-009	HIGH	SQL Injection in /register Allows Unauthenticated Arbitrary Database Writes	CWE-89	A03:2025	A.8.8	ID.SC-2	Req 6.3.3
VUL-010	HIGH	SQL Injection in db.raw_query() — Table Name and Filter Values Both Unparameterized	CWE-89	A03:2025	A.8.8	ID.SC-2	Req 6.3.3
VUL-011	HIGH	Path Traversal in /download Allows Unauthenticated Arbitrary File Read	CWE-22	A01:2025	A.8.2, A.8.3	PR.AC-4	Req 7
VUL-012	HIGH	MD5 Used for Password Hashing Without Salt — Trivially Crackable via Rainbow Tables	CWE-916	A02:2025	A.8.9	PR.IP-1	Req 2
VUL-013	HIGH	Cryptographically Weak PRNG Used for Password Reset Tokens — Brute-Forceable Token Space	CWE-338	A07:2025	A.8.5	PR.AC-7	Req 8.3
VUL-014	HIGH	Hardcoded JWT Secret Key in Source Code Enables Token Forgery	CWE-321	A02:2025	A.8.9	PR.IP-1	Req 2
VUL-015	HIGH	Hardcoded Stripe Production Secret Key in Source Code	CWE-798	A07:2025	A.8.5	PR.AC-7	Req 8.3
VUL-016	HIGH	Hardcoded Database Credentials (admin/admin123) in Source Code	CWE-798	A07:2025	A.8.5	PR.AC-7	Req 8.3

ID	SEVERITY	FINDING	CWE	OWASP 2025	ISO 27001:2022	NIST CSF	PCI DSS V4
VUL-017	HIGH	Flask Debug Mode Enabled by Default — Werkzeug Interactive Console Exposed	CWE-1188	A05:2025	A.8.28	PR.DS-5	Req 6.2.4
VUL-018	HIGH	PyYAML 5.3.1 Contains Critical RCE Vulnerability (CVE-2020-14343, CVSS 9.8)	CWE-502	A06:2025	A.8.25	PR.IP-2	Req 6.2
VUL-019	MEDIUM	Insecure Direct Object Reference (IDOR) in /user/<user_id> Enables Cross-User Data Access	CWE-639	A01:2025	A.8.2, A.8.3	PR.AC-4	Req 7
VUL-020	MEDIUM	Plaintext Passwords and Reset Tokens Written to Application Logs	CWE-532	A09:2025	A.8.15	DE.AE-3	Req 10.2
VUL-021	MEDIUM	Wildcard CORS Policy Allows Cross-Origin Requests from Any Domain	CWE-942	A05:2025	A.8.28	PR.DS-5	Req 6.2.4
VUL-022	MEDIUM	Plaintext HTTP Used for Internal Service Communication — No TLS or Certificate Validation	CWE-319	A02:2025	A.8.9	PR.IP-1	Req 2
VUL-023	MEDIUM	No Request Size Limits — Application Vulnerable to Memory Exhaustion DoS	CWE-770	A05:2025	A.8.28	PR.DS-5	Req 6.2.4
VUL-024	LOW	Database Connection Leak in bulk_insert() — Connection Never Explicitly Closed	CWE-404	A05:2025	A.8.28	PR.DS-5	Req 6.2.4

08 Security Strengths & Observations

The following controls were found to be correctly implemented and should be preserved during remediation efforts.

✓ **Parameterized Placeholders in bulk_insert()**

The bulk_insert() function in db.py uses '?' placeholder parameters for row values in INSERT statements, preventing SQL injection for the values being inserted. This demonstrates awareness of parameterization — a pattern that should be extended to all database interactions in the application.

✓ **Environment Variable Loading for Database and Redis URLs**

DATABASE_URL and REDIS_URL in config/settings.py are correctly loaded from environment variables with sensible defaults, demonstrating awareness of 12-factor app config principles. This pattern should be extended to all secrets including AWS keys, JWT secret, Stripe key, and Flask session secret.

✓ **Session-Based Authentication Mechanism Present**

The /user/<user_id> endpoint checks for session['user_id'] before processing, showing that an authentication pattern exists and is applied in at least one location. This foundation needs to be extended to all sensitive endpoints and strengthened against session forgery via a proper secret key.